

Cost, Pricing & Billing

Every paid service Agently depends on, what each one charges us, and exactly how that becomes a number on a tenant's wallet.

Prepared 5 September 2026 **Margin** 70% gross (3.333× cost) **Model** Prepaid wallet, pay-as-you-go

This document contains internal cost and margin.

Tenants see a single charge per unit of usage. They never see what a service costs us, nor the multiplier applied. Vendor rates below are published list prices retrieved on 4 September 2026 and are what we *pay*, not what we charge.

1. How a tenant is billed

Agently is prepaid and usage-based. There is no monthly subscription and no seat fee. A tenant buys credit, and each unit of work draws that credit down as it happens. New accounts start with **\$5 of free credit**; a card must be on file before a phone number can be bought or a chatbot put live, so the account cannot run up cost and walk away.

The pricing formula

$$\text{tenant charge} = \text{our cost} \div (1 - 0.70) = \text{our cost} \times 3.333$$

70% is *gross margin*, not a 70% uplift. A service that costs us \$1.00 is charged at \$3.33, of which \$2.33 is gross profit. The margin is stored in `billing_platform_settings.default_target_margin_percent` and is adjustable from the admin panel; the same `100 ÷ (100 - margin)` form is used everywhere so per-event and periodic charges can never disagree.

Two kinds of cost

KIND	WHAT IT IS	HOW THE TENANT PAYS FOR IT
DIRECT	Metered per tenant by the vendor: a call minute, a synthesised character, a token. Attributable to one tenant with certainty.	Charged immediately , as the event happens. The wallet is debited within the same request that logs the usage.
SHARED	Flat bills that arrive regardless of tenant activity: the Lightsail container services, the Supabase plan. No per-tenant figure exists.	Never billed as a line item. It is a cost of doing business, priced into the unit rate. See section 4.

Why infrastructure is not billed separately.

It was built both ways and measured. Apportioning the flat server bill across tenants and charging it at 3.33× produced a **\$78.89 monthly infrastructure charge** for the busiest tenant, in a month where *every tenant combined* consumed \$17.67 of actual provider cost. Infrastructure would have been 78% of the bill.

Vapi, Bland and Twilio all absorb infrastructure into the per-unit price; none of them itemise a customer's share of their servers. Agently now does the same. The apportionment engine still runs — it is what tells us whether the unit rate is high enough (section 4).

2. Direct costs — what each vendor charges us

List prices retrieved 4 September 2026. Our configured rates live in the `billing_rate_cards` table and are seeded from `VENDOR_RATE_CARD_JSON` — this table is the vendor's public price, which those rates should be reconciled against.

Twilio — telephony DIRECT

ITEM	UNIT	OUR COST	TENANT PAYS (×3.333)
Inbound local voice	per minute	\$0.0085	\$0.0283
Outbound local voice	per minute	\$0.0140	\$0.0467
Inbound toll-free voice	per minute	\$0.0220	\$0.0733
Outbound toll-free voice	per minute	\$0.0140	\$0.0467
Local number rental	per month	\$1.15	\$3.83
Toll-free number rental	per month	\$2.15	\$7.17

Source: [twilio.com/en-us/voice/pricing/us](https://www.twilio.com/en-us/voice/pricing/us). US rates; other countries differ and are charged at the vendor's country rate.

OpenAI — reasoning and realtime voice DIRECT

MODEL	UNIT	OUR COST	TENANT PAYS (×3.333)
gpt-realtime — audio in	per 1M tokens	\$32.00	\$106.67
gpt-realtime — audio out	per 1M tokens	\$64.00	\$213.33
gpt-realtime-mini — audio in	per 1M tokens	\$10.00	\$33.33
gpt-realtime-mini — audio out	per 1M tokens	\$20.00	\$66.67
gpt-4o — text in	per 1M tokens	\$2.50	\$8.33
gpt-4o — text out	per 1M tokens	\$10.00	\$33.33
gpt-4o-mini — text in	per 1M tokens	\$0.15	\$0.50
gpt-4o-mini — text out	per 1M tokens	\$0.60	\$2.00

Source: [developers.openai.com/api/docs/pricing](https://openai.com/api/docs/pricing). Realtime audio is the dominant cost of a voice call — see the worked example in section 3.

ElevenLabs — speech synthesis DIRECT

PLAN	MONTHLY	CREDITS	COST / 1K CHARS	TENANT PAYS / 1K
Starter	\$6	30,000	\$0.20	\$0.667
Creator	\$11	121,000	\$0.18	\$0.600
Pro	\$99	600,000	\$0.17	\$0.567
Scale	\$299	1,800,000	\$0.17	\$0.567
Business	\$990	6,000,000	\$0.17	\$0.567

Source: elevenlabs.io/pricing. Note this is a *subscription with an included allowance*, not pure per-unit billing. The effective per-character cost therefore depends on how much of the allowance we consume; the rate card should carry the blended rate for our actual plan, not the list rate.

Resend — transactional email DIRECT

PLAN	MONTHLY	INCLUDED	OVERAGE / 1K	TENANT PAYS / 1K
Free	\$0	3,000	—	—
Pro	\$20	50,000	\$0.90	\$3.00
Pro (larger)	\$35	100,000	\$0.90	\$3.00
Scale (from)	\$90	100,000	\$0.90 → \$0.46	\$3.00 → \$1.53

Source: resend.com/pricing.

3. What a call actually costs

A worked example, so the margin is legible against a real unit of work. A **five-minute inbound local voice call** handled by the realtime model, with roughly 3,000 characters of synthesised speech:

COMPONENT	QUANTITY	OUR COST	TENANT PAYS
Twilio inbound local	5 min @ \$0.0085	\$0.0425	\$0.1417
OpenAI realtime audio in	~15K tokens @ \$32/M	\$0.4800	\$1.5998
OpenAI realtime audio out	~7K tokens @ \$64/M	\$0.4480	\$1.4931
ElevenLabs synthesis	3K chars @ \$0.17/1K	\$0.5100	\$1.6998
Total for one 5-minute call		\$1.4805	\$4.9344

Read this number carefully.

Token counts above are **illustrative** — they are a plausible shape for a five-minute call, not a measurement. Real consumption varies with how much the caller talks, model choice, and whether ElevenLabs or the realtime model produces the audio. The *rates* are published vendor prices; the *quantities* must be read off `billing_usage_events` for real calls before this table is used to set customer-facing expectations.

The practical implication: a \$5 free-credit account can make roughly **one five-minute realtime call** before needing to top up. If the trial is meant to feel more generous than that, the lever is either a cheaper model for trial accounts (`gpt-realtime-mini` is ~3× less) or a larger grant via `BILLING_SIGNUP_GRANT_USD`.

4. Shared infrastructure — measured, not billed

Two bills arrive as a single flat number no matter what tenants do. Neither vendor can say which tenant caused what, because for these services the question has no answer: one set of containers runs everyone's calls.

What we actually run

Verified 5 September 2026 from the AWS API (`lightsail get-container-services` and `get-container-service-powers`), not from an estimate.

SERVICE	POWER	SCALE	\$/NODE/MO	MONTHLY
agently-calls	small-1 — 0.5 vCPU, 1 GB	2	\$15.00	\$30.00
agently-ingest	nano-1 — 0.25 vCPU, 0.5 GB	1	\$7.00	\$7.00
AWS Lightsail total				\$37.00

Data transfer in and out cost **\$0.00** — 1.16 GB in and 1.18 GB out in August, far inside the bundle allowance. Actual metered Lightsail spend was **\$7.16 in August** (services were created on 26 August, so a partial month) and is tracking to roughly \$31–37/month at the current scale.

SERVICE	BILLING SHAPE	MONTHLY
Supabase SHARED	Plan base plus overage: disk \$0.125/GB past 8 GB, egress \$0.09/GB past 250 GB, storage \$0.0213/GB past 100 GB. Database is currently 126 MB, 13 of 60 connections in use.	\$10.00 standard tier
Total flat infrastructure		\$47.00

Supabase usage is not fully measurable by API.

Database size and connection count are readable; bandwidth/egress, MAU, storage objects and edge-function invocations are not exposed by the Management API. The authoritative figure is the org usage page, and `SUPABASE_MONTHLY_USD` must be updated from it when overages appear:

supabase.com/dashboard/org/etnncnjhkskqiwrxcwd/usage?projectRef=qozbmfwxlcuvwzbxtnnn

The apportionment algorithm

Infrastructure is not billed to tenants, but it is still measured per tenant. That measurement is what proves the unit rate is high enough, and it is what a per-customer profitability question gets answered with.

Splitting a shared bill by one single metric would be a guess — split by call minutes and a chat-only tenant with a large knowledge base contributes nothing to the database they fill. So each bill is attributed by the usage that actually drives *that* bill:

BILL	ATTRIBUTED BY	WHY
Lightsail	100% compute seconds	Containers running websocket audio. Time spent processing a live session is what consumes them.
Supabase	60% stored bytes 40% activity (USD of work)	The plan is roughly compute against disk and egress, so the split follows that shape.

$$\text{tenant's share of a bill} = \sum (\text{driver weight} \times \text{tenant's share of that driver})$$

Four properties the algorithm guarantees, each covered by a test:

- **The split always sums to 100%.** No cost silently disappears between tenants.
- **A driver nobody used redistributes its weight.** If no tenant stored anything this period, storage's 60% flows to the remaining drivers rather than dropping real cost on the floor.
- **Idle tenants attract nothing.** Share is measured in magnitude of work, not row counts — a tenant who sent one email is attributed \$0.00, not a slice of the server bill.
- **Cost nobody can be attributed is reported, not hidden.** If nothing happened all period, the whole bill shows as absorbed rather than as a misleading zero.

The loss guard

Absorbing infrastructure creates one specific danger: the unit rate looks like it carries a 70% margin while the flat server bill quietly eats it, and nothing on any invoice would reveal that. Every cycle therefore computes:

FIGURE	MEANING
True cost	provider cost + infrastructure cost
Revenue	what tenants were actually charged
Actual margin	margin after infrastructure, not the headline 70%
Break-even multiplier	multiple of provider cost at which we stop losing money
Required multiplier	multiple needed to hit the target margin with infrastructure absorbed — this is the number that should set unit prices

Current reading: we are running at a loss.

Measured against live data for the 30 days to 5 September 2026, at \$37 Lightsail + \$10 Supabase:

Provider cost, all tenants	\$17.67
Infrastructure cost	\$46.36
True cost	\$64.03
Revenue at the current 3.333×	\$58.91
Gross profit	−\$5.12

Actual margin is **−8.69%**, not 70%. Infrastructure is **72.4%** of what we spend. Break-even is **3.623×** provider cost; we charge 3.333×

This is a *volume* problem, not a rate problem. \$47/month of fixed cost divided by very little usage dominates everything. The same 3.333× rate is comfortably profitable once usage grows — at roughly \$650/month of provider cost it yields about 68% actual margin, with infrastructure falling to under 7% of spend. The two levers are growing usage or trimming the container scale; raising the rate to the arithmetic 12× that would force 70% margin at today's volume is not a real option.

If a dedicated customer ever needs it

The engine keeps a `passthrough` mode that bills each tenant their apportioned infrastructure share directly. It is off, and it should stay off for self-serve customers; it exists for a large dedicated deployment where the customer expects to see infrastructure itemised. Set it with `BILLING_INFRA_MODE=passthrough`.

5. Current status

OFF The billing engine is not running on a schedule.

In absorb mode it never charges anyone, so the risk of enabling it is reporting noise rather than a wrong invoice. It is still off until deliberately switched on:

#	STEP	STATUS
1	Apply <code>migrations/billing_period_charges.sql</code> (the audit table)	Not applied
2	Set <code>AWS_LIGHTSAIL_MONTHLY_USD=37</code> and <code>SUPABASE_MONTHLY_USD=10</code> (verified, section 4)	Unset in <code>.env</code>
3	Set <code>BILLING_ENGINE_ENABLED=true</code> and schedule <code>npm run billing:cycle:commit</code> hourly	Unset — in absorb mode this only writes audit rows, it charges nobody

Until all three are done the engine reports and charges nothing. Running `npm run billing:cycle` at any time prints exactly what *would* be charged, writing nothing.

Every charge is reconstructable.

Each period writes one row per tenant to `billing_period_charges` holding the raw driver measurements, the per-provider cost breakdown, the apportionment share, the margin used and the multiplier. A disputed charge can be recomputed from first principles months later — a figure alone is not an answer to a customer asking why.

6. Open items

ITEM	DETAIL
Rate cards vs list prices	Rates in <code>billing_rate_cards</code> were not read for this document (production database). They should be reconciled against the vendor prices in section 2.
Subscription vendors	ElevenLabs and Resend bill a monthly allowance, not pure per-unit. Their effective rate depends on how much of the allowance we use, so a flat per-character rate over- or under-charges depending on volume.
Supabase overage	Only the plan base is configured, and the Management API does not expose bandwidth, MAU or storage-object usage. Read them off the org usage page and update <code>SUPABASE_MONTHLY_USD</code> as overages appear.
Margin below target	Actual margin is negative at current volume (section 4). Re-check the loss guard as usage grows; it turns positive without any rate change.
Call cost quantities	Section 3 token counts are illustrative. Measure real calls from <code>billing_usage_events</code> before quoting these figures to customers or using them for trial sizing.